Software Test Plan (STP) - College Timetable Generator

Project: College Timetable Generator
Version: 1.0
Authors: Harshitha B. T, Diya B R, Gaurav S, Harsha K R
Date: 27-10-2026
Status: Final draft for submission

Project Details

Field	Details
Project Title	College Timetable Generator
Project Type	Web-based software application
Project Domain	Education / academic timetable scheduling
Programming Language	Python
Framework	Flask
Testing Framework	pytest
Version Control	Git and GitHub
CI Tool	GitHub Actions (.github/workflows/ci.yml)
Repository URL	https://github.com/harshithabt/college-timetable-generator
Deployment Platform	Render (render.yaml, Procfile, gunicorn)

1. Introduction

Purpose: This document defines the objectives, scope, strategy, environment, schedule, responsibilities and reporting approach for testing the College Timetable Generator. Testing aims to verify that the application generates valid timetables, respects scheduling constraints, rejects invalid input, handles impossible scheduling requests safely, and works correctly through its web interface.

Scope: Testing covers CSV and manual input of subject offerings, timetable generation using a backtracking scheduler, teacher/class/room conflict prevention, teacher availability, classroom assignment, timetable display, edit-and-regenerate behaviour, the automated pytest suite, the GitHub Actions CI workflow and the Render deployment. Penetration testing, real university system integrations and hosting-provider internals are excluded.

References: GitHub repository **(https://github.com/harshithabt/college-timetable-generator);** source files app.py, app/scheduler.py, app/constraints.py, templates/; automated tests tests/test_scheduler.py and tests/test_flask_app.py; test datasets in test_data/ (14 CSV files); CI workflow .github/workflows/ci.yml; deployment files render.yaml, Procfile, requirements.txt.

Definitions: CSV (Comma-Separated Values), CI (Continuous Integration), UI (User Interface), RTM (Requirements Traceability Matrix), UAT (User Acceptance Testing), STP (Software Test Plan).

2. Test Items

Test item	Description
Scheduling module (app/scheduler.py)	Backtracking generator that assigns day/slot/room to every required period
Constraint module (app/constraints.py)	Detects teacher, class and room clashes and validates a finished timetable
Flask application (app.py)	Routes /, /generate, /edit, /subjects-template.csv; form and CSV validation; classroom assignment
Web interface (templates/)	Input form, results timetable grid, error messages
Test datasets (test_data/)	Valid, impossible and invalid CSV scenarios
Automated test suite	pytest unit and Flask route/integration tests
CI workflow	GitHub Actions job that installs dependencies and runs python -m pytest -v
Deployment configuration	Render web service started with gunicorn app:app, health check path

3. Features to be Tested

Features mapped to requirement IDs:

Requirement ID	Feature / expected behaviour
TT-F-001	Accept valid subject offerings through the manual form or CSV upload (columns: course_code, subject_name, teacher, class_id, periods_per_week)
TT-F-002	Generate a complete timetable when the supplied constraints are satisfiable
TT-F-003	A teacher is never scheduled in two places in the same time slot
TT-F-004	A class section never has overlapping subjects
TT-F-005	Optional teacher availability (teacher, day, slot) is respected
TT-F-006	Classrooms are assigned without double-booking a room
TT-F-007	Validation of empty files, header-only files, missing or wrong headers, blank required fields and invalid period values
TT-F-008	Impossible scheduling constraints are reported without returning a conflicting timetable
TT-F-009	Settings can be edited and the timetable regenerated
TT-F-010	Clear, understandable validation and scheduling-failure messages are shown
TT-NF-001	The automated test suite runs reliably locally and in GitHub Actions CI
TT-NF-002	Invalid or impossible requests return a result or error within a reasonable time (no hanging request)
TT-DEP-001	The application starts on Render with gunicorn and the health check path / responds

4. Features Not to be Tested

- Integration with real university ERP, identity management or production databases (not implemented)
- Processing of real institutional data not supplied to the project
- Formal security certification or penetration testing
- Large-scale performance and load guarantees
- Internal infrastructure of GitHub Actions and Render

5. Test Approach / Strategy

Levels:

- Unit tests - scheduler and constraint functions (tests/test_scheduler.py)
- Integration tests - Flask routes, form/CSV handling and scheduler working together (tests/test_flask_app.py)
- System tests - full workflow through the web UI using the CSV datasets
- Acceptance tests (UAT) - demonstration of the final application to the faculty reviewer
- Deployment smoke test - live Render application loads and completes a representative timetable runTypes:
- Functional testing of generation and constraints
- Negative / input-validation testing with invalid and impossible datasets
- Regression testing - the full pytest suite is re-run after every change
- CI testing - verification of GitHub Actions runs on pushes and pull requests
- Basic usability checks of forms and error messages

Entry Criteria: Latest source is in the GitHub repository; dependencies install from requirements.txt; test datasets are available; expected outcomes are defined.

Exit Criteria: Every planned test case has a recorded status (passed, failed or blocked); critical defects affecting timetable generation are resolved or documented; no timetable that violates a hard constraint is ever accepted; required CI checks pass; the live Render application is verified before deployment is called complete.

5.1 Security Validation

- Malformed, empty or non-UTF-8 uploads are rejected with a controlled error message and no stack trace
- Uploaded data is validated before use (headers, required fields, numeric values)
- Debug mode and internal paths are not exposed by the deployed application
- Long-running requests caused by impossible input are checked, since they can tie up the server (see TT-NF-002)
- Full penetration testing is outside this plan

5.2 Test Data / CSV Dataset Catalogue and Execution Results

All datasets are in test_data/ and were run through the real /generate route on 27-10-2026 (main branch, Python 3.12, default settings of 5 days x 6 slots unless stated). In automatic classroom mode the app requires at least one available room per class section, so each scenario was run with enough rooms for its sections.

ID	Dataset	Scenario	Expected result	Observed result	Status
01	01_normal_timetable.csv	Valid, normal input	Timetable generated	Generated (HTTP 200)	Pass
02	02_teacher_availability.csv	Availability restrictions	Teacher placed only in permitted slots	One teacher limited to slots 1-4 each day: valid timetable, 0 availability violations	Pass
03	03_limited_classrooms.csv	Limited room resources	No room clash; clear failure if rooms are insufficient	4 rooms for 8 sections rejected with a clear message; with 8 rooms generated, no clash	Pass
04a	04a_impossible_teacher_overload.csv	Teacher workload exceeds available slots (one teacher has 40 periods, only 30 slots exist)	Prompt schedulingfailure message; no teacher clash	No response within 60 s	**Fail (DEF01)**
04b	04b_impossible_class_overload.csv	Section has more periods than slots (CSE-A has 40 periods, 30 slots exist)	Prompt schedulingfailure message; no section clash	No response within 90 s	**Fail (DEF01)**
04c	04c_impossible_room_capacity.csv	Room availability constraint (15 sections)	Honour the implemented room rule; fail if no valid assignment	With 4 rooms rejected with a clear message; with 15 rooms generated, no clash (about 25 s)	Pass (see DEF-02)
05a	05a_invalid_empty_file.csv	Empty CSV	Validation error	"The uploaded CSV is empty." (HTTP 400)	Pass
05b	05b_invalid_header_only.csv	Header row only	No-data error	"The subject list contains no data rows." (HTTP 400)	Pass
05c	05c_invalid_missing_column.csv	Missing required column	CSV rejected with useful message	Header-requirement message listing the required columns (HTTP 400)	Pass
05d	05d_invalid_blank_subject_name.csv	Blank subject name	Row rejected	"Subject row 1: subject name is required." (HTTP 400)	Pass
05e	05e_invalid_values.csv	Invalid periods (0, negative, text)	Invalid values reported	"Periods per week ... must be a positive whole number." (HTTP 400)	Pass
05f	05f_invalid_blank_required_fields.csv	Blank required fields	Missing values reported	"Subject row 1: course code is required." (HTTP 400)	Pass

05g	05g_invalid_duplicate_rows.csv	Duplicate rows	Behaviour matches the application's duplicate rule; no corruption	Accepted (HTTP 200); duplicate rows receive unique internal IDs	Pass (see DEF-02)
05h	05h_invalid_wrong_headers.csv	Wrong column names	CSV rejected with header error	Header-requirement message (HTTP 400)	Pass

6. Test Environment

Hardware: Developer laptops (Windows) for local testing; Render-hosted web service for deployment testing.

Software: Python 3.12 (as used by the CI workflow); Flask 3.1+, pytest 8+, gunicorn 22+ (from requirements.txt); web browser for manual UI checks.
Tools: pytest; Git and GitHub; GitHub Actions (CI); Render dashboard (deployment and logs); browser developer tools.

Test Data: The 14 CSV datasets in test_data/ plus manually entered subject rows.

7. Test Schedule

Milestones:

#	Activity	Status
1	Review requirements, input format and test datasets	Completed
2	Run unit and Flask route/integration tests	Completed - 70 passed
3	Execute the 14 CSV scenarios through the web application	Completed 27-10-2026; results in section 5.2
4	Verify GitHub Actions CI runs (push, pull request)	In progress - attach run link
5	Configure and verify Render deployment, run live smoke test	In progress - attach live URL
6	Retest after fixing DEF-01 and prepare final test summary	Before submission

8. Test Deliverables

- Software Test Plan (this document)
- CSV test datasets and dataset catalogue (test_data/)
- Automated test scripts (tests/)
- Test execution log (pytest output and CSV scenario results)
- Defect reports (section 14)
- GitHub Actions run links or screenshots
- Render deployment evidence and live URL
- Final test summary report

9. Roles and Responsibilities

Role	Name	Responsibility
Scheduling	Harshitha B T	Scheduling algorithm, testing and CI; fixes schedulerrelated defects
Web application	Diya B R	Flask web application, UI and frontend; fixes UI and form defects
Data and deployment	Gaurav S	Data Validating; Render deployment; coordinates CSV and deployment checks
Data Testing	Harsha K R	Data model, sample data, integration tests
Project team	All members	Review test results, prioritise defects, approve readiness for demonstration

10. Risks and Mitigation

Risk	Mitigation
Impossible input makes the backtracking search run for a very long time	Add up-front feasibility checks (teacher and class periods versus available slots) and retest datasets 04a and 04b
Input data does not match the expected CSV schema	Verify headers and required fields; run datasets 05a-05h
Invalid input causes an uncontrolled failure	Execute negative tests and confirm a controlled 400 response with a clear message
Scheduling conflicts appear in generated output	Validate every result with the constraint checker for teacher, class and room clashes
Code changes break existing features	Run the full pytest suite and CI on every push
CI passes but deployment fails	Run a separate live smoke test on Render after each deployment
Incorrect production startup command	Verify gunicorn app:app and the / health check in Render logs
Conflicting team changes	Use feature branches, pull requests and code review

11. Assumptions & Dependencies

- The latest project code, requirements.txt, tests and datasets are available in the GitHub repository
- CSV files use the header course_code, subject_name, teacher, class_id, periods_per_week
- In automatic classroom mode each class section needs its own available room
- Python dependencies install in the local, CI and Render environments
- GitHub Actions has the permissions needed to run the workflow
- Deployment depends on a valid Render configuration and a successful build

12. Suspension & Resumption Criteria

Suspend testing if:

- The application fails to start or a critical error blocks most test cases
- Required datasets, dependencies or the test environment are unavailable
- A blocking defect prevents execution of related test cases- The Render build or deployment fails (deployment testing only) Resume testing when:
- Blocking defects are fixed and the application starts successfully
- Datasets, dependencies and environment configuration are restored
- Failed test cases are re-run after the fix is merged into main, followed by regression testing

13. Test Case Management & Traceability

RTM ensures mapping of requirements to datasets and test areas:

Requirement	**Related test scenario(s)**	**Evidence**	**Status**
TT-F-001	01 normal CSV; manual form tests in test_flask_app.py	pytest output; upload result	Pass
TT-F-002	01_normal_timetable.csv	Generated timetable, constraint check	Pass
TT-F-003	02; 04a; scheduler unit tests	No overlapping teacher assignment	Pass (04a see DEF-01)
TT-F-004	04b; scheduler unit tests	No overlapping section assignment	Pass (04b see DEF-01)
TT-F-005	02_teacher_availability.csv	Slots compared with availability settings	Pass
TT-F-006	03; 04c	Room allocation and clash check	Pass
TT-F-007	05a-05h	Validation messages; test assertions	Pass
TT-F-008	04a, 04b, 04c	**Failure response and no conflicting schedule**	**Fail (DEF-01)**

TT-F-009	Edit/regenerate tests in test_flask_app.py	Settings restored; updated output	Pass
TT-F-010	05a-05h; 03	Error messages in responses	Pass
TT-NF-001	pytest locally; GitHub Actions run	Workflow run URL and job status	Pass locally; attach CI link
TT-NF-002	04a, 04b	Response time of impossible requests	**Fail (DEF-01)**
TT-DEP-001	Render deployment and health check	Render logs; live URL; smoke test	To be recorded

14. Test Metrics & Reporting

Metrics collected:
- Total planned test cases and number executed, passed, failed, blocked
- Execution percentage and pass percentage
- Defects found, by severity, and defects resolved
- Requirement coverage
- CSV scenarios executed
- GitHub Actions and Render deployment statusCurrent results (27-10-2026):
- Automated suite (python -m pytest -v, main branch, Python 3.12): 70 passed, 0 failed
- CSV scenarios: 14 of 14 executed; 12 passed, 2 failed (04a, 04b) - Execution: 100%; pass rate for CSV scenarios: 86% (12/14) Defects identified:

ID	Severity	Description	Steps to reproduce	Expected	Actual
DEF-01	High	Impossible workloads are not rejected up front, so the request does not return	Upload 04a (teacher with 40 periods) or 04b (section with 40 periods) with enough rooms, 5 days x 6 slots	Quick error such as "teacher/section needs more periods than available slots"	No response within 60-90 s
DEF-02	Low	Dataset 04c is not impossible in the application (the app has no per-room capacity; it checks total room-slots), and exact duplicate rows (05g) are accepted	Upload 04c with 15 rooms; upload 05g	Documented rule or rejection	04c generates after about 25 s; 05g generated with unique internal IDs

Reporting: A per-run status log is kept and a final test summary separates verified results from blocked or unexecuted checks. Results are re-run and refreshed before submission. Reports:
- Per-run execution status log
- Final Test Summary Report

15. Approvals

Role	Name	Signature / Date
QA Lead		
Dev Lead		
Product Owner		